Name :- Jai Pratap Rajput

CU ID : CU24250071

BTech CSE 3rd Year SECTION - "C"

Roll No. :- 27

==========

Lab Sheet No. 3

```
import pandas as pd
import numpy as np
from scipy import stats
import statsmodels.api as sm
import statsmodels.formula.api as smf
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
```

```
df = pd.read_csv("/content/titanic_cleaned.csv")
print("Dataset shape:", df.shape)
print(df.head())
```

```
Dataset shape: (775, 22)
   survived  pclass     sex   age  sibsp  parch     fare  class    who  \
0         0       3    male  22.0      1      0   7.2500  Third    man
1         1       1  female  38.0      1      0  71.2833  First  woman
2         1       3  female  26.0      0      0   7.9250  Third  woman
3         1       1  female  35.0      1      0  53.1000  First  woman
4         0       3    male  35.0      0      0   8.0500  Third    man

   adult_male  ... sex_encoded  alive_encoded  embarked_C  embarked_Q  \
0        True  ...           1              0       False       False
1       False  ...           0              1        True       False
2       False  ...           0              1       False       False
3       False  ...           0              1       False       False
4        True  ...           1              0       False       False

   embarked_S  age_scaled  fare_scaled  family_size    age_group  \
0        True    0.371556     0.098747            2  Young Adult
1       False    0.647039     0.970901            2        Adult
2        True    0.440427     0.107941            1  Young Adult
3        True    0.595386     0.723239            2  Young Adult
4        True    0.595386     0.109644            1  Young Adult

   fare_category
0            Low
1           High
2            Low
3           High
4            Low

[5 rows x 22 columns]
```

```
df.columns = df.columns.str.strip()
```

```
print("Shape of dataset:", df.shape)
print("\nColumns:")
print(df.columns)

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())
```

```
       'family_size', 'age_group', 'fare_category'],
      dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 775 entries, 0 to 774
Data columns (total 22 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   survived       775 non-null    int64
 1   pclass         775 non-null    int64
 2   sex            775 non-null    object
 3   age            775 non-null    float64
 4   sibsp          775 non-null    int64
 5   parch          775 non-null    int64
 6   fare           775 non-null    float64
 7   class          775 non-null    object
 8   who            775 non-null    object
 9   adult_male     775 non-null    bool
 10  alive          775 non-null    object
 11  alone          775 non-null    bool
 12  sex_encoded    775 non-null    int64
 13  alive_encoded  775 non-null    int64
 14  embarked_C     775 non-null    bool
 15  embarked_Q     775 non-null    bool
 16  embarked_S     775 non-null    bool
 17  age_scaled     775 non-null    float64
 18  fare_scaled    775 non-null    float64
 19  family_size    775 non-null    int64
 20  age_group      775 non-null    object
 21  fare_category  775 non-null    object
dtypes: bool(5), float64(4), int64(7), object(6)
memory usage: 106.8+ KB

Missing Values:
survived         0
pclass           0
sex              0
age              0
sibsp            0
parch            0
fare             0
class            0
who              0
adult_male       0
alive            0
alone            0
sex_encoded      0
alive_encoded    0
embarked_C       0
embarked_Q       0
embarked_S       0
age_scaled       0
fare_scaled      0
family_size      0
age_group        0
fare_category    0
dtype: int64
```

```
if 'age' in data.columns:
    data['age'] = data['age'].fillna(data['age'].median())

if 'fare' in data.columns:
    data['fare'] = data['fare'].fillna(data['fare'].median())
```

```
if 'embarked' in data.columns:
    data['embarked'] = data['embarked'].fillna(
        data['embarked'].mode()[0]
    )

print("Missing values after cleaning:")
print(data.isnull().sum())
```

```
Missing values after cleaning:
survived         0
pclass           0
sex              0
age              0
sibsp            0
parch            0
fare             0
class            0
who              0
adult_male       0
alive            0
alone            0
sex_encoded      0
alive_encoded    0
embarked_C       0
embarked_Q       0
embarked_S       0
age_scaled       0
fare_scaled      0
family_size      0
age_group        0
fare_category    0
dtype: int64
```

```
numeric_columns = ['age', 'fare', 'sibsp', 'parch']

print("MEAN")
print(data[numeric_columns].mean())

print("\nMEDIAN")
print(data[numeric_columns].median())

print("\nMODE")
print(data[numeric_columns].mode().iloc[0])

print("\nVARIANCE")
print(data[numeric_columns].var())

print("\nSTANDARD DEVIATION")
print(data[numeric_columns].std())

print("\nCOMPLETE DESCRIPTIVE STATISTICS")
print(data[numeric_columns].describe())
```

```
MEAN
age      29.363123
fare     26.593137
sibsp     0.529032
parch     0.420645
dtype: float64

MEDIAN
```

```
age      28.0
fare     15.9
sibsp     0.0
parch     0.0
dtype: float64

MODE
age      28.00000
fare     73.41975
sibsp     0.00000
parch     0.00000
Name: 0, dtype: float64

VARIANCE
age      174.546333
fare     523.178676
sibsp      0.980745
parch      0.706550
dtype: float64

STANDARD DEVIATION
age      13.211598
fare     22.873099
sibsp     0.990326
parch     0.840565
dtype: float64

COMPLETE DESCRIPTIVE STATISTICS
              age        fare       sibsp       parch
count  775.000000  775.000000  775.000000  775.000000
mean    29.363123   26.593137    0.529032    0.420645
std     13.211598   22.873099    0.990326    0.840565
min      0.420000    0.000000    0.000000    0.000000
25%     21.000000    8.050000    0.000000    0.000000
50%     28.000000   15.900000    0.000000    0.000000
75%     36.000000   34.197900    1.000000    1.000000
max     58.500000   73.419750    8.000000    6.000000
```

```
print("Survival Count:")
print(data['survived'].value_counts())

plt.figure(figsize=(6, 4))

sns.countplot(x='survived', data=data)

plt.title("Titanic Survival Count")
plt.xlabel("Survived (0 = No, 1 = Yes)")
plt.ylabel("Number of Passengers")

plt.show()
```

```
Survival Count:
survived
0    455
1    320
Name: count, dtype: int64
```

```
print("Survival by Gender:")

gender_survival = pd.crosstab(
    data['sex'],
    data['survived']
```

```
)

print(gender_survival)

plt.figure(figsize=(7, 5))

sns.countplot(
    x='sex',
    hue='survived',
    data=data
)

plt.title("Survival by Gender")
plt.xlabel("Gender")
plt.ylabel("Number of Passengers")

plt.show()
```

```
Survival by Gender:
survived    0    1
sex
female     76  216
male      379  104
```

```
correlation_columns = [
    'survived',
    'pclass',
    'age',
    'sibsp',
    'parch',
    'fare'
]

correlation_matrix = data[correlation_columns].corr()

print("Correlation Matrix:")
print(correlation_matrix)
```

```
Correlation Matrix:
          survived    pclass       age     sibsp     parch      fare
survived  1.000000 -0.331388 -0.075582 -0.037841  0.069864  0.306633
pclass   -0.331388  1.000000 -0.345319  0.087050  0.038998 -0.741242
age      -0.075582 -0.345319  1.000000 -0.284784 -0.187100  0.147240
sibsp    -0.037841  0.087050 -0.284784  1.000000  0.379535  0.263081
parch     0.069864  0.038998 -0.187100  0.379535  1.000000  0.238917
fare      0.306633 -0.741242  0.147240  0.263081  0.238917  1.000000
```

```
plt.figure(figsize=(8, 6))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)

plt.title("Titanic Correlation Matrix")
plt.show()
```

```
correlation, p_value = stats.pearsonr(
    data['age'],
    data['fare']
)

print("Pearson Correlation:", correlation)
print("P-value:", p_value)
```

```
Pearson Correlation: 0.1472404630896287
P-value: 3.874614238193193e-05
```

```
survived_age = data[data['survived'] == 1]['age']

not_survived_age = data[data['survived'] == 0]['age']

# Perform t-test
t_statistic, t_pvalue = stats.ttest_ind(
    survived_age,
    not_survived_age,
    equal_var=False
)

print("T-TEST RESULTS")
print("----------------------")
print("T-statistic:", t_statistic)
print("P-value:", t_pvalue)
```

```
T-TEST RESULTS
----------------------
T-statistic: -2.080481156501386
P-value: 0.037869963180922256
```

```
alpha = 0.05

if t_pvalue < alpha:
    print("\nDecision: Reject H0")
    print("There is a significant difference in average age.")
else:
    print("\nDecision: Fail to Reject H0")
    print("There is no significant difference in average age.")
```

```

Decision: Reject H0
There is a significant difference in average age.
```

```
class_1 = data[data['pclass'] == 1]['fare']

class_2 = data[data['pclass'] == 2]['fare']

class_3 = data[data['pclass'] == 3]['fare']

# Perform ANOVA
f_statistic, anova_pvalue = stats.f_oneway(
    class_1,
    class_2,
    class_3
)
```

```
print("ANOVA RESULTS")
print("----------------------")
print("F-statistic:", f_statistic)
print("P-value:", anova_pvalue)
```

```
ANOVA RESULTS
----------------------
F-statistic: 573.9214015737475
P-value: 1.9042732700319744e-153
```

```
alpha = 0.05

if anova_pvalue < alpha:
    print("\nDecision: Reject H0")
    print("There is a significant difference in fare among passenger classes.")
else:
    print("\nDecision: Fail to Reject H0")
    print("There is no significant difference in fare among passenger classes.")
```

```

Decision: Reject H0
There is a significant difference in fare among passenger classes.
```

```
plt.figure(figsize=(8, 5))

sns.boxplot(
    x='pclass',
    y='fare',
    data=data
)

plt.title("Fare Distribution by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Fare")

plt.show()
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
import matplotlib.pyplot as plt


X = data[['age']]


y = data['fare']


regression_model = LinearRegression()

regression_model.fit(X, y)


y_pred = regression_model.predict(X)
s
print("Linear Regression Results")
print("------------------------")
```

```
print("Coefficient:", regression_model.coef_[0])
print("Intercept:", regression_model.intercept_)


r2 = r2_score(y, y_pred)

print("R² Score:", r2)
```

```
Linear Regression Results
-------------------------
Coefficient: 0.2549158425601123
Intercept: 19.10801177038479
R² Score: 0.021679753970848314
```

```


r2 = r2_score(y, y_pred)

print("R² Score:", r2)
```

```
R² Score: 0.021679753970848314
```

```
plt.figure(figsize=(8, 5))

plt.scatter(
    data['age'],
    data['fare'],
    alpha=0.5
)

plt.plot(
    data['age'],
    y_pred
)

plt.xlabel("Age")
plt.ylabel("Fare")

plt.title("Linear Regression: Age vs Fare")

plt.show()
```

```
X_stats = data[['age']]

X_stats = sm.add_constant(X_stats)

y_stats = data['fare']

model = sm.OLS(y_stats, X_stats).fit()


print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                   fare   R-squared:                       0.022
Model:                            OLS   Adj. R-squared:                  0.020
Method:                 Least Squares   F-statistic:                     17.13
Date:                Wed, 02 Sep 2026   Prob (F-statistic):           3.87e-05
Time:                        12:35:31   Log-Likelihood:                -3516.4
```

```
No. Observations:                 775   AIC:                             7037.
Df Residuals:                     773   BIC:                             7046.
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         19.1080      1.983      9.636      0.000      15.215      23.001
age            0.2549      0.062      4.139      0.000       0.134       0.376
==============================================================================
Omnibus:                      103.381   Durbin-Watson:                   2.037
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              145.082
Skew:                           1.054   Prob(JB):                    3.13e-32
Kurtosis:                       2.786   Cond. No.                         78.6
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specifi
```

```
print("\n========== FINAL CONCLUSION ==========")

if t_pvalue < 0.05:
    print("1. T-Test: There is a statistically significant difference")
    print("   in the average age of survivors and non-survivors.")
else:
    print("1. T-Test: There is no statistically significant difference")
    print("   in the average age of survivors and non-survivors.")


if anova_pvalue < 0.05:
    print("2. ANOVA: Passenger class has a significant effect on fare.")
else:
    print("2. ANOVA: No significant difference in fare was found")
    print("   among passenger classes.")


print("3. Pearson Correlation between Age and Fare:",
      round(correlation, 4))


print("4. R² Score of Age vs Fare Regression:",
      round(r2, 4))
```

```
========== FINAL CONCLUSION ==========
1. T-Test: There is a statistically significant difference
   in the average age of survivors and non-survivors.
2. ANOVA: Passenger class has a significant effect on fare.
3. Pearson Correlation between Age and Fare: 0.1472
4. R² Score of Age vs Fare Regression: 0.0217
```

```
Start coding or generate with AI.
```